

Waiting and Blocking in a Linux Driver



This is the second article on waiting and blocking in a Linux driver. As we've discovered earlier, waiting and blocking are as inevitable in Linux drivers as death and taxes in real life, so let us try to understand the process better with this insightful article.

In the last article, we implemented a basic wait mechanism in the Linux driver and as stated, such an implementation is very primitive and prone to bugs. Also, the process was being put to an unconditional sleep, which is obviously not the case in real life scenarios. So, how do we make the process wait on an event? Read on to dive further into the implementation of wait mechanisms in Linux drivers.

Waiting/blocking on an event

In real life scenarios, a process never waits without any event. The event can be a specified period before which the device is available for use. It could even be the arrival of data from some device such as the disk. Or the event could very well be waiting for some shared resource to be available. Since we don't have any physical event to wait upon, let's simulate the event by using a global variable *flag*. Shown below is the code snippet for this:

```
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/fs.h>
#include <linux/cdev.h>
#include <linux/device.h>
#include <linux/uaccess.h>
#include <linux/sched.h>

#define FIRST_MINOR 0
#define MINOR_CNT 1

static dev_t dev;
static struct cdev c_dev;
static struct class *cl
static struct task_struct *sleeping_task;
static char flag = 'n';

int open(struct inode *inode, struct file *filp)
{
    printk(KERN_INFO "Inside open \n");
    return 0;
}

int release(struct inode *inode, struct file *filp)
{
    printk(KERN_INFO "Inside close \n");
    return 0;
}

ssize_t read(struct file *filp, char *buff, size_t count,
loff_t *offp)
{
    printk(KERN_INFO "Inside read\n");
    sleeping_task = current;
    slp:
    if (flag != 'y')
    {
        printk(KERN_INFO "Scheduling out\n");
        set_current_state(TASK_INTERRUPTIBLE);
        schedule();
    }
    if (flag == 'y')
        printk(KERN_INFO "Woken up\n");
    else
    {
        printk(KERN_INFO "Interrupted by signal\n");
        goto slp;
    }
    flag = 'n';
    return 0;
}

ssize_t write(struct file *filp, const char *buff, size_t
count, loff_t *offp)
{
    printk(KERN_INFO "Inside Write\n");
    if (copy_from_user(&flag, buff, 1) != 0)
    {
        printk(KERN_INFO "Failed to read from user space\n");
    }
    wake_up_process(sleeping_task);
    return count;
}

struct file_operations fops = {
    .read = read,
    .write = write,
    .open = open,
    .release = release
};
```